

強化学習：インプットノート

目的

- ・収益の最大化

手段

- ・状態に応じて行動を選択する

教師あり学習との類似点・相違点

ー 類似点

教師あり学習

- ・データに対して、推論を行い、正解率を高める

強化学習

- ・データに対して、方策を選択して、収益を高める

ー 相違点

① 状態 → 行動 → 状態 → 行動

このように変化していく中で、強化学習は状態が変化していく。

そのため、状態には相関が発生してくる。

② 収益がわかるまで時間差がある。

状態 → 行動 → 状態 → 行動の流れの中で、一回目の行動を行った直後は、まだその行動が収益に対しての良し悪しが不明瞭。

強化学習の学習部分

ー 教師ありの場合

データ → ラベル 正解率

→ 部分をモデルで学習

ー 強化学習の場合

状態(1) → 行動(2) → 収益 (3) → $= (1) + (2)$

強化学習では、3つの → を学習できる。

ただ、それぞれで目的の細部が変わるため、学習の仕方も変わってくる。

代表例

(1) → : PN

(2) → : QL

(3) → : VN

=====

強化学習の全体像

State: 状態 Action: 行動 Reward: 報酬 g: 収益

rの合計がgとなる場合と、囲碁や将棋のように $g=r$ =勝ち・負け・引き分けとなる場合がある。

$s \rightarrow a$ のループの中でaに対するrが出てくる。これの合計がgとなる。

ポイント

(方策) (policy: π)

$s \rightarrow a$ に関しては、自分たちで制御可能となる部分。

(MDR: マルコフ決定過程) (環境)

$a \rightarrow s$ や $a \rightarrow r$ に関しては、結果となり、自分たちで制御が不可能

そのため、 $a \rightarrow s$ と $a \rightarrow r$ に関しては、データの収集を行うことで保管

これにより、学習を通じた行動選択をみだすことができる。

=> この全体像を裏で支えているのが、価値観数 (V 、 Q)

=====

(MDR: マルコフ決定過程) (環境) について。

$s \rightarrow a \rightarrow r, s \rightarrow a \rightarrow r, s$ のループの動きの中で、 $a \rightarrow s$ 、 $a \rightarrow r$ の部分がマルコフ決定過程 (環境) の部分となる。

MDP の構成要素: 5 つ

① 状態の集合 S

全体集合 S に対して、各場面での状態 s が部分集合となる

$s \in S$

② 行動の集合 A_s

全体集合 S の部分集合 S に対して、集合 A_s が定めている。

すなわち、全体集合 A_s と部分集合 a を行動という。

$a \in A_s$

③ 状態推移の確率: 行動 (a) $\rightarrow$ 状態 (s)

行動に応じて次の状態が決まる。

$P(s' | s, a)$

条件付き確率となる。

状態に対する行動でどのように状態になるかは常に決まっていないため、状態 1 になる確率 50%、状態 2 になる確率 30%、状態 3 になる確率 20% のように扱う。

=> 状態 $s \in S$ 、 $a \in A_s$ に対して S 上の確率分布 $P(s' | s, a)$ が定まっている。

④ 報酬の確率: 行動 (a) $\rightarrow$ 報酬 (r)

行動に応じて報酬が決まる。① と似たもの

$P(r | s, a)$

ポイント

s については、常に同じ a を選択したとしても、その時々で s の状態は変化し、一定でない。

=> 状態 $s \in S$ 、 $a \in A_s$ に対して R の確率分布 $P(r | s, a)$ が定まっている。

⑤ 初期状態の確率: $p(S_0)$

S 上の確率分布 $p(S_0)$ が定まっている。

囲碁や将棋においては、初期が常に一定であるが、7 並べや UNO のようなものを強化学習の題材にする場合は、最初の状態がその都度違う。これに対応するのが $p(S_0)$ となる。

=====

(方策) (policy: π)

強化学習において、収益最大化のため状態に応じた行動の選択する。

この、「状態に応じた行動の選択」の部分が方策に該当。

方策の定式化

状態 s と行動 a を入力すると、状態 s で行動 a を選択する確率を与える確率分布として $\pi(a | s)$ と書く

ポイント：なぜ関数 $a = f(s)$ でなく、確率として扱うのか。

① マルコフ決定過程 MDP も確率のため、そこと合わせている。

② 一番大事：探索 → この S の時は、この a をとるのが最善そうとわかっているけど、他にもっといい選択肢があるかもしれない。そのため、最善の行動を取る確率を 95%、ランダム性を持たせた行動をとる確率を 5% と設定して、より良いものを探索していくため。

③ 行動が連続値の場合に対応するため。

囲碁・将棋のように A が離散的な場合は関係ないが、ロボット制御の電圧を例にすると、電圧 1.00V ピッタリでなくても、1.00 から ± 0.25 であれば結果同じ a になるとする場合は、その a は確率として扱うと幅が広がるため。

価値関数

① エピソードと軌跡

これまでの S, A, R や π, MDP があると、

【初期状態】 【ステップ 1】 【ステップ 2】

$s_0, a_0, r_0 \rightarrow s_1, a_1, r_1 \rightarrow s_2, a_2, r_2 \dots$

この流れとなり、各時間ステップにおける s, a, r はすべて確率であるため、全体を通して、確率分布を獲得できる。

この初期状態 $\rightarrow$ 時間ステップ 1 $\rightarrow$ ステップ 2 $\dots$ END

全体のことをエピソードといい、このエピソードが集まったものが、軌跡となる。

例：将棋の場合、ある 1 局がエピソード。将棋協会全体の対局記録が軌跡となる。ただ、このエピソードと、軌跡の定義は曖昧。

② 収益と価値関数

収益 $g = \sum r_t$ と定義できたがこのままでは、 g の最大化を求める上で発散してしまう可能性があるため、収束させてあげる。

まず、期待値に変換

$E[G] = E[\sum R_t] = \sum E[R_t]$ となる。：期待収益

これをさらに変形

これから行うことのモチベーションの確認

収益 $g = \sum r_t$ と定義できたがこのままでは、 g の最大化を求める上で発散してしまう可能性があるため、収束させてあげる。

その収束手段がいくつかある。1, 平均してあげる。2, 割引率 γ の適用 ($0 \leq \gamma \leq 1$)

その変形例 ↓

(1)平均報酬型

$$E[G] = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T E[R_t]$$

(2)期待割引収益

$$E[G] = \sum_{t=0}^{\infty} \gamma^t E[R_t] \quad (0 \leq \gamma \leq 1)$$

(2.1)

$$V^\pi = E[G \mid S_0 = S] : \text{状態価値観数}$$

(2.2)

$$Q^\pi = E[G \mid S_0 = S, A_0 = a] : \text{行動価値観数}$$

=>この4つを最大化

↑の4つをより詳細に見ていく。その上での前提条件↓

全体の記号について：

- ・ s_t, a_t, r_t : 時刻 t での状態、行動、報酬の値
- ・ S_t, A_t, R_t : 時刻 t での状態、行動、報酬の確率変数。

確率変数：確率によって値が決まるが、確率によって決まった特定の値ではなく確率だけが与えられた状態。(大文字)

- ・ S, A, R : 集合

=====

- ・ 期待割引収益についての深掘り

収束させる手段として $(0 \leq \gamma \leq 1)$ を採用。

なぜ、収束するのか $\gamma = 1/2$ の例で確認。

$$G = R_0 + \frac{1}{2}R_1 + \frac{1}{4}R_2 + \frac{1}{8}R_3 + \dots$$

となり将来の報酬に小さい値をかけるため、発散しにくくなる (=収束しやすくなる)。

λ を採用すると、計算において収束しやすくなることはわかった。ただ、いきなり出てきた γ をほんとに採用していいのかの理由についても見ていく。

- ・ 理由1：うまく学習できればいい。

実際に、 γ をタスクに合わせていい感じに設定すると学習がうまく行った。

また、 γ の値と、収益を計算する上での r_t の t の数値のなんとなくの関わりもわかっている。

例1： $\gamma = 0.9$ の時

$E[G]$ の最大化 $\sim t=10$ までの報酬。

例2： $\gamma = 0.99$ の時

$E[G]$ の最大化 $\sim t=100$ までの報酬

これは、行うタスクによってどれだけのステップ分の報酬を収益に加えるかわ変わってくる。囲碁や将棋の場合は、報酬と収益が最後に勝ち負けで決まるため、 $\gamma = 1$ となる。

- ・ 理由2：経済的に自然

割引現在価値の考えを、強化学習のステップにも当てはめた方がいいという考え。

この期待割引収益が一般的な収益になる。

=====

価値関数

- 価値関数とは、

$E[G] = \sum \gamma^t E[R_t]$ ($0 \leq \gamma \leq 1$) : 収益

$V^\pi(s) = E[G \mid S_0 = s]$: 状態価値関数

$Q^\pi(s, a) = E[G \mid S_0 = s, A_0 = a]$: 行動価値関数

説明

$E[X \mid *]$ は、* の場合の X の期待値を表す。

・ 状態価値関数 V^π は、初期状態が s の場合の収益

→ s の後もらえる報酬の合計の期待値

=> 状態価値関数 V^π は、状態 s の良さを表す。

・ 行動価値関数 Q^π は、初期状態 s で行動 (方策: π) をとった後の報酬の合計の期待値

=> 行動価値関数 $Q^\pi(s, a)$ は、状態 s での行動 (方策) a の良さを表す。

- 価値観数の技法

ステップが $0 \rightarrow 1 \rightarrow 2 \dots$ と続く時

各ステップに対応する s, a がある。

これらの s, a に対応する、状態価値関数と行動価値観数がある。

すなわち

=> 方策 π の学習と並行して価値関数 V^π と Q^π も学習すると効率が上がる。

また、タスクによって片方のみを一緒に学習させるといった選択肢が出てくる。

=====

Generalized Policy Iteration : 学習方法の話

確認: 強化学習の流れ

$S \rightarrow a \rightarrow s \rightarrow a \rightarrow s \dots$ この繰り返し。

① 最初の $s \rightarrow a$ の部分が方策となる。② その後の $a \rightarrow s$ が MDP でのデータとなり、これを蓄積していく。③ これを学習して行動選択の改善

この3つの流れを繰り返していく。このループを Generalized Policy Iteration という。

=> すなわち、方策評価と方策改善の繰り返し

ここで、方策改善は、Policy Improvement といいパラメータを変えて π を改善していく。

また、方策評価は、Policy Evaluation といい、価値関数 V^π 、 Q^π の推定 (学習) を指す

(強化学習の難しさ: 次につながるポイント)

1, 部分問題が多い

・ データから π を学習する場合

・ データと V, Q から π を学習する場合

・ π とデータから価値観数 V, Q を学習する場合など

何を使って、何を学習するのかの部分問題が多い。

2,注力場所が異なる

成果物として考えたときに、Qの学習しかしない手法がまずある。

これだけでも、各状態sで $Q(s, a)$ が最大になる行動aを選択すればよくなる。

この他にもQしか学習品手法、 π しか学習しな手法と色々あり、難しさへつながる。

↓これを受け

強化学習の4つの対象

1,data(データ), 2, π (方策), 3,V,q(価値関数) 4, p,r(MDP)

この4つを駆使して学習していく。

1 Q-learning

(1)方策 π とMDRを使ってdataを集める。

(2)今集まっているdataと V,Q 価値関数を使って、V,Q 価値関数をアップデートする。

この部分がQ-learningと言われている。

(3)これができたら、アップデートされたV,Q 価値関数を使って、方策をアップデート。

(4)これを繰り返す。

まとめとポイント

・今後の流れを捉えるうえで、何を使って何を学習するかを捉える。あとは、基本この作業を繰り返してくだけ。

・今後はその学習方法 (V,Q 価値関数をアップデートなど) について見ていくことになる。

=====

ベルマン方程式

ベルマン方程式は2つある。

- ベルマン期待方程式
 - ベルマン最適方程式

=====

ベルマン方程式の大枠

Step①：状態の次は行動

(1)まず $V^{\pi}(s)$ ：sの時の収益 →次に行動へ

(2)行動aは確率 $\pi(a | s)$ で選択される。

(3)行動aの後の収益は、 $Q^{\pi}(s, a)$

すなわち、行動aの種類がありその選択は確率によってなされる。また、各行動aに対してaの収益である $Q^{\pi}(s, a)$ があることから、

ポイント： $V^{\pi}(s) = \sum \pi(a | s) * Q^{\pi}(s, a)$ となる。

=> これにより、状態sからその1歩先がわかるようになった。

Step②：行動の次は、報酬と次の状態

(1) まず、 $Q^{\pi}(s, a)$ で s の状態から a のした後の収益 → 次に報酬と状態

(2) 報酬：確率 $p(r | s, a)$ で報酬 r がもらえる。 状態：確率 $p(s' | s, a)$ で次の状態 s' になる。

すなわち

$$\begin{aligned} \Rightarrow Q^{\pi}(s, a) &= (\text{はじめての報酬 } R_0 \text{ の期待値}) + (S_1 = S' \text{ 以降の収益}) \\ &= \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^{\pi}(s') \end{aligned}$$

このステップ(1)と(2)を踏まえると $\Rightarrow$ ベルマン期待方程式へ

・ベルマン期待方程式

$$\begin{aligned} V^{\pi}(s) &= \sum \pi(a | s) * Q^{\pi}(s, a) \\ &= \sum \pi(a | s) * \{ \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^{\pi}(s') \} \end{aligned}$$

$$\begin{aligned} Q^{\pi}(s, a) &= \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^{\pi}(s') \\ &= \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * \sum \pi(a' | s') * Q^{\pi}(s', a') \end{aligned}$$

=====

ベルマン最適方程式

・概要

- いい感じの MDP には最適方策という方策があり、すべての状態行動について価値関数が最大となる。この最適な方策を π^* とし、強化学習での究極の目標は、最適方策の発見となる。(基本的には、この最適方策は存在する)

- π^* の方策をとった時の状態価値関数、行動価値関数をそれぞれ V^* と Q^* とする。

これを元に、ベルマン方程式を見ると次のようになる。

$$\begin{aligned} V^* &= \max(\sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^*(s')) \\ Q^* &= \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * \max Q^*(s', a') \end{aligned}$$

ベルマン期待方程式同様に Step(1)(2)で考える。

Step(1)：状態の次は行動

$$V^*(s) = \max_a Q^*(s, a)$$

これは、ベルマン期待方程式の Step(1)の $V^{\pi}(s) = \sum \pi(a | s) * Q^{\pi}(s, a)$ より

今回は、最適方策 π^* のみで考える。この場合、 $\max_a Q^{\pi}(s, a)$ となり、そのみを選択することから、選択する確率は $100\% = 1$ となるため。

Step(2)：行動から状態

$$Q^*(s, a) = \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^*(s')$$

ここでも、ベルマン期待方程式の Step(2)の $\sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^{\pi}(s')$ より、行動から状態において、そもそも方策が関係する部分は、 $V^{\pi}(s')$ の部分。他の、 $\sum p(r | s, a) * r + \gamma \sum p(s' | s, a)$ の部分は、方策ではなく MDP の部分いなるため変化なし。

このステップ(1)と(2)を踏まえると ==> ベルマン最適方程式へ

ベルマン最適方程式

$$V^* = \max(\sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^*(s'))$$

$$Q^* = \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * \max Q^*(s', a')$$

=====

アルゴリズム 1

方策反復法 : Policy iteration

ベルマン方程式をベルマン作用素を使ってグリグリ

グリグリ内容↓

1, 概要

- ・ やること : 方策評価と方策更新の繰り返しで最適方策 π^* を得る
- ポイント : 方策評価 $\rightarrow$ 価値関数の推定、最適方策 $\pi^* \rightarrow$ 価値関数が最大の方策
=> 多くの強化学習アルゴリズムが、方策評価と方策更新を繰り返す。
- ・ 前提 : MDP が既知の場合に使える
- $P(r | s, a)$ や $p(s' | s, a)$ などが既知
 - 例 : ボードゲームの AI 作り。反対にレコメンド強化学習とかでは使えない
- ・ 状態数が大きすぎない時に利用可
- 状態数の行列積を繰り返す
- ・ data 集めはせず、MDP から π^* を直接得る。
- 方策によるデータ集めをせず、MDP から π^* を直接得る

ベルマン作用素

$V(s)$: 状態価値関数 (ホンモノ)

$V^{\wedge}(s)$: 状態価値関数の推定値

ベルマン作用素 : ベルマン方程式を用いて $V^{\wedge*}(s)$ の精度を高める技

$\rightarrow$ すなわち、 $V(s)$ がすぐわかればいいが、わからないので $V^{\wedge}(s)$ の推定値の精度を上げていって、ホンモノに近づけること

2, 方策評価

モチベーション :

方策評価とは、価値関数の推定。今回は状態価値関数 $V(s)$ を推定

状況 :

- ・ MDP は既知 ($p(r | s, a)$, $p(s' | s, a)$)

1 区切りまとめ

Policy iteration の目的は、状態価値関数 $V(s)$ を推定する。

前提は、MDP は既知である。

状態価値関数 $V(s)$ の推定方法は、以下

$$V(s) = \sum \pi(a | s) * \{ \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^{\pi}(s') \}$$

この時、 $V(s)$ は連立 1 時方程式で求めることができる。

→ ということか

仮に状態が s_1, s_2 の 2 つの場合。

$$V(s_1) = \sum \pi(a | s_1) * \{ \sum p(r | s_1, a) * r + \gamma \sum p(s_1 | s_1, a) * V^{\pi}(s_1) + p(s_2 | s_1, a) * V^{\pi}(s_2) \}$$

$$V(s_2) = \sum \pi(a | s_2) * \{ \sum p(r | s_2, a) * r + \gamma \sum p(s_2 | s_2, a) * V^{\pi}(s_2) + p(s_1 | s_2, a) * V^{\pi}(s_1) \}$$

この時、MDP が既知であることから、式を簡略化すると

$$x = a + bx + cy$$

$$y = d + ex + fy$$

(A, b, c, d, e, f は定数)

→ こうすれば、 x, y での連立 1 次方程式 として解を出せる。

これが、状態が 100 個あっても、解答できることがわかる。

また、今後推定されたていく、状態価値関数は、 $V^{\pi}(s)$ と表す。

更新

状況：

- ・ $p(r | s, a), p(s' | s, a)$ は既知である。
- ・ 状態価値関数の推定値 $V^{\pi}(s)$ がある。
- ・ より収益の大きい方策 π_{new} が欲しい。

=> $\pi_{\text{new}} = \arg \max Q^{\pi}$ で OK

・ ここで Q^{π} は、行動価値関数の推定値。ベルマン方程式の Step2 で対応

$$Q^{\pi}(s, a) = \sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^{\pi}(s')$$

・ また、 $\arg \max$ の役割としては、状態 S では $Q^{\pi}(s, a)$ が最大となる行動 a を選択する方策。

ベルマン作用素

$V(s)$ ：状態価値関数 (ホンモノ)

$V^{\pi}(s)$ ：状態価値関数の推定値

ベルマン作用素：ベルマン方程式を用いて $V^{\pi}(s)$ の精度を高める技

→ すなわち、 $V(s)$ がすぐわかればいいが、わからないので $V^{\pi}(s)$ の推定値の精度を上げていって、ホンモノに近づけること

具体的にベルマン作用素は、 $V^{\pi}(s)$ {今ある推定値} から、以下の式で $V^{\text{new}}(s)$ {新しい推定値} を作る変換

$$V^{\text{new}}(s) = \sum \pi(a | s) (\sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^{\pi}(s'))$$

すると誤差が γ 倍減る具体的には ↓

$$\text{Max} | V^{\text{new}}(s) - V^*(s) | \leq \gamma \text{Max} | V^{\pi}(s) - V^*(s) |$$

まとめ：**Policy iteration**

【状態1】：初期

方策： π (init)

価値関数： V, Q (init)

マルコフ： p, r (既知)

【状態2】：方策評価

初期の3要素とベルマンで、 $V^{\wedge}, Q^{\wedge}$ の推定

【状態3】：方策更新

$\pi = \operatorname{argmax} Q^{\wedge} \pi$ で新たな方程式の獲得

これを繰り返す

=====

アルゴリズム2

価値反復法：Value iteration

ベルマン最適方程式をベルマン最適作用素を使ってグリグリ

グリグリ内容↓

目的：

- ・最適状態価値関数 V^* の推定値 $V^{\wedge*}$ を計算
- ・ $V^{\wedge*}$ から $Q^{\wedge*}$ を計算し、 $\pi = \operatorname{argmax} Q^{\wedge*}$ で最適方策 π^* を得る。

使える前提：方策反復法と同様

- ・ MDP は既知
- ・ 状態数少なめ (囲碁・将棋とかでも多い)
- ・ data は不要

インプットの流れ

ベルマン方程式 → ベルマン最適方程式の流れのように進める

・学習ステップ1

ベルマン最適作用素

$V^*(s)$ ：最適状態価値関数 (ホンモノ)

$V^{\wedge*}(s)$ ：最適状態価値関数の推定値

ベルマン作用素：ベルマン方程式を用いて $V^{\wedge*}(s)$ の精度を高める技

ベルマン最適作用素は、 $V^{\wedge*}(s)$ に以下の式を施して $V^{\wedge*new}(s)$ を計算する変換

$$V^{\wedge*new} = \max(\sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^*(s'))$$

これにより、誤差が γ 倍になる。具体的には↓

$$\operatorname{Max}|V^{\wedge*new}(s) - V^*(s)| \leq \gamma \operatorname{Max}|V^{\wedge*}(s) - V^*(s)|$$

これにより、 $\gamma < 1$ ならベルマン最適作用素を何度も適用すれば、 V^* の誤差の小さな推定値 V^* が得られる。

=> 精度が十分高くなったら、 V^* の遺産は完了して、次のステップへ

・学習ステップ2

ベルマン最適方程式の Step2 より $Q^*(s, a)$ を計算

$$Q^*(s, a) = \sum p(r | s, a)r + \gamma \sum p(s' | s, a) V^*(s)$$

・学習ステップ3

$\pi = \operatorname{argmax} Q^*$ より

状態 s で $Q^*(s, a)$ が最大の行動 a を選択する方策

まとめ

【状態1】：初期

価値関数： V, Q (init)

マルコフ： p, r (既知)

【状態2】：方策評価

初期の3要素とベルマンで、 V^*, Q^* の推定

【状態3】：方策更新

$\pi = \operatorname{argmax} Q^\pi$ で新たな方程式の獲得

価値反復法は、方策反復法と違い繰り返さない。

理由：探索をなわずに進んでるため。

=====

アルゴリズム3

TD法

ポイント：MDRがわからなくても使えて、データを買って学習するアルゴリズム

TD法のざっくり概要

- 方策評価（状態価値関数の推定値 $V^\pi(s)$ の精度を高める）に特徴のあるアルゴ
 - データ収集を伴う（ベルマン作用素）のデータ近似を行う → MDRをらなくても行える
 - MDRを知らなくても行える
 - 状態が聞けてもOK → データ効率が良いため

TD誤差

ポイントとなる数式： $V^{\pi_{\text{new}}} = V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s))$

状況：

- ・MDRがあるのである

- ・ 方策 π を用いて集めたデータがある
- ・ 状態価値関数 $V^\pi(s)$ の推定値 $V^\pi(s)$ の精度を上げたい。

まず、データについて、どうやって集まったか。どのようなデータがあるか見ていく

・ データ

これまでの、 $s \rightarrow a \rightarrow r \rightarrow s \rightarrow a \rightarrow r \dots$ と続けていった。

すなわち、方策 π と MDR で状態・行動・報酬のデータがられる。

また、この1連の流れをエピソードとして使った。

対して、TD 法では s, a, r, s' を1組として使う。

これを使ってTD 法を行う。

- ・ ポイントとなる数式： $V^{\pi_{\text{new}}} = V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s))$ について
 → やっていることは、データ s, a, r, s' を使って推定値 $V^\pi(s)$ を $V^{\pi_{\text{new}}}$ に更新する
 - $r + \gamma V^\pi(s') - V^\pi(s) : \delta \text{TD 誤差}$
 - γ は $(0 \leq \gamma \leq 1)$
- 数式から、 $V^\pi(s)$ から $r + \gamma V^\pi(s') - V^\pi(s)$ へ α だけ寄せた量

・ さらに $r + \gamma V^\pi(s)$ を紐解く

ベルマン要素の式： $\sum \pi(a | s)(\sum p(r | s, a) * r + \gamma \sum p(s' | s, a) * V^\pi(s'))$ がベース

今回は、MDR は未知のため、ベルマン作用素は使えない。がデータを大量に使って近似させることで、 V^* の精度を高める。データ量を無限にした時、 α は徐々に小さく ($\alpha \rightarrow 0$)。

・ TD 法の特徴

① PI と VI との比較

PI と VI は、MDR が既知のため、ベルマン作用素・ベルマン最適作用素を使って、すべての状態 s についての推定値 $V^\pi(s)$ を更新できる。← 期待値を MDR で計算するため。

ー ただ、TD の場合は、MDR がのため、データ s, a, r, s' を使った時は

1つの状態 s の推定値 $V^\pi(s)$ のみ更新する。これをいろんなデータで繰り返す。

② データ効率がいい

$V^{\pi_{\text{new}}} = V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s))$ より

$V^\pi(s')$ は、一個前のデータを使ったものとなり、これを次のデータ、次のデータと引き継いで流れになる。すると、1番最初のデータをずっと使うこととなるため、データ率が良い。

③ 変なデータに引っ張られる

1部に変なデータがると、そのデータもりかえし使うので、それに引っ張られることになる。

(ブートストラッピング)

=====

Q 学習 : データを使った学習の全体像に迫る

① : Q 学習とは

- TD 誤差 (の 1 種) を扱う
 - Generalized Policy Iteration の代表格
- 方策評価 (価値関数の推定) と方策更新を繰り返すアルゴ
- Off-policy $\leftarrow$ on もりそれが次に出てくる σ のため、この違いはそこまで触れない
- TD 法と同様に MDR は未知でデータ収集を伴う。

②Q 学習の全体像

Step(0) : 方策 π と Q^* の初期化

- Q^* は、最適行動関数 Q^* の推定値
- 初期化 : なんでも良いから値を振る

Step(1) : データ集め

方策 π に従って s, a, r, s' を集める。

MDR が未知のため、 s おいて方策の a をしたらどんな r で、どんな s' になるかのデータをとる。

Step(2) : 方策評価 (価値関数のアップデート)

$\sigma(q)$ を用いて $Q^*(s, a)$ をアプデ

Step(3) : 方策更新

$\pi_{\text{new}} = \epsilon\text{-greedy} Q^*$ で新方策を作る

この Step(1) から (3) を繰り返すと

$\Rightarrow Q^* \rightarrow Q^*$ で最適方策 $\pi^* \arg\max Q^*$ が手に入る

③Q 学習の TD 誤差

ポイントとなる数式 :

$$Q^*_{\text{new}}(s, a) = Q^*(s, a) + \alpha \sigma(q)$$

$$\sigma(q) = r + \gamma \max_{a'} Q^*(s', a') - Q^*(s, a)$$

流れは TD 法のバルマン最適作用素の右辺と一致していて・・・って流れで一緒。

これで、精度の高い価値関数がくるから、これを使って方策を更新する。

この手法 : $\pi_{\text{new}} = \epsilon\text{-greedy} Q^*$

- この方策は、状態 s で確率 $1-\epsilon$ で $Q^*(s, a)$ が最大の行動 a を選択。確率 ϵ で全行動からランダムに選択する (探索)。 ϵ は小さい値。

なぜ、最適作用素なのにランダム性を入れるのか

$\rightarrow$ MDR が未知のため、いろんなデータを収支しないと本当にこれが出てきで圧と言えないため。

=====

SARSA

概要

- ・ TD 誤差の 1 種を扱う
- ・ Generalized Policy Iteration の代表格
- 方策評価 (価値関数の推定) と方策更新を繰り返すアルゴ
 - ・ on-policy
 - ・ Q 学習同様に、MDR は未知で、データ収集が伴う。

Step0 : π と Q^π の初期化

- 行動価値関数 Q^π の推定値
- Step1 : π を用いて、データ収集
- 今回のデータ収集では、 **s, a, r, s', a'** を 1 つのデータセットとして扱う。
- Step2 : データを用いて Q^π を更新
- Step3 : $\pi_{\text{new}} = \epsilon\text{-greedy}Q^*$ で方策を更新

TD 誤差

- ・ データ : s, a, r, s', a' を用いて $Q^\pi(s, a)$ を更新していく
- ・ 更新式 : $Q^{\pi_{\text{new}}}(s, a) = Q^\pi(s, a) + \alpha(r + \gamma Q^\pi(s', a') - Q^\pi(s, a))$
- 更新式の中身は、Q 学習同様に、ベルマン作用素の右辺
 - α はの役割は、良さそうな方向にちょっとずつ動かすことで、変なデータに引っ張られない。

Q 学習と SARSA の差異

Q 学習 :

- ・ データは、 s, a, r, s' の 4 つ。
- ・ ベルマン最適作用素によって、 $\max Q^*(s', a')$ を用いる
- ϵ による探索も入れながらも、最適なものを選択していく。

SARSA :

- ・ データは、 s, a, r, s', a' の 5 つ
- ・ ベルマン最適作用素を用いるが、 $\max Q^*(s', a')$ を必要としない。
- これは、データからなる確かな a' を使うことで、学習が安定する。
 - 対して、Q 学習では最も良い方策に一直線で辿り着く可能性はある。

方策更新については、Q 学習同様に

$\rightarrow \pi_{\text{new}} = \epsilon\text{-greedy}Q^*$

=====

N-step TD 法、N-step Q 学習、N-step SARSA

・ N-step TD 法

1 : TD 法の復習 (N-step TD 法も同様)

– TD 法の発展

- 方策評価のアルゴ
- MDR は未知
- データ収集を伴う
- 状態数が多くても OK
- ポイントとなる数式 : $V^{\pi_{\text{new}}} = V^{\pi}(s) + \alpha(r + \gamma V^{\pi}(s') - V^{\pi}(s))$

2 : N-step TD 法 (2-step TD 法)

TD 法における数式 : $V^{\pi_{\text{new}}} = V^{\pi}(s) + \alpha(r + \gamma V^{\pi}(s') - V^{\pi}(s))$ より、 $V^{\pi}(s')$ をステップさせて、 $V^{\pi}(s+2)$ とすると

$$V^{\pi_{\text{new}}} = V^{\pi}(s) + \alpha(r + \gamma(r(t+1) + \gamma V^{\pi}(s+2)) - V^{\pi}(s))$$

3 : N-step

1 の $V^{\pi}(s')$ の部分をさらに 3step、4step・・・と深掘りしていくと、
TD 法における数式 : $V^{\pi_{\text{new}}} = V^{\pi}(s) + \alpha(r + \gamma V^{\pi}(s') - V^{\pi}(s))$ より、
 $r + \gamma V^{\pi}(s')$ の部分が : $r(t) + \gamma r(t+1) + \gamma^2(r(t+2) + \gamma^3(r(t+3) + \dots + V^{\pi}(s+n) - V^{\pi}(s))$ となる。

・ N-step の特徴

： メリット

$V^{\pi}(s')$ の部分を N-step 先まで深掘り、かく step での期待値に対して、 γ^n が適用されるため、めっちゃ正確になっていく

： デメリット

メリットで挙げためっちゃ正確なのは期待値。すなわち、全体の方策評価は、いい方向には進んでいくが、期待値の足し算となるため、分散が大きくなっていく点が懸念点

・ N-step Q 学習と N-step SARSA

変形は、TD→2Step→N-step と同様。

・ N-step Q 学習 :

$$Q^{\pi_{\text{new}}}(s, a) = Q^{\pi}(s, a) + \alpha \sigma(q)$$

$\sigma(q) = r + \gamma \max_{a'} Q^{\pi}(s', a') - Q^{\pi}(s, a)$ より、

$\max_{a'} Q^{\pi}(s', a')$ の部分が深ぼられていく

・ N-step SARSA

$$Q^{\pi_{\text{new}}}(s, a) = Q^{\pi}(s, a) + \alpha(r + \gamma Q^{\pi}(s', a') - Q^{\pi}(s, a))$$

$Q^{\pi}(s', a')$ の部分を深ぼる形にしたいだけ

N-stepのポイント：n-step分, y^n が追加される。

=====

TD (λ) : N-step 発展型

ベースはTD 法やN-stepTD 法と同様

- N-stepTD 法の発展
 - 方策評価のアルゴ
 - MDR は未知
 - データ収集を伴う
 - 状態数が多くても OK

N-stepの問題点：N個先の状態を含めることができるが、最適なNを探るのがめんどい
解答→全てのNを採用する。その式が↓

$$V^{\pi_{\text{new}}}(\text{st}) = V^{\pi}(\text{st}) + \alpha \sigma^{\lambda}(t)$$
$$\sigma^{\lambda}(t) = (1-\lambda)(\sigma^1(t) + \lambda \sigma^2(t) + \dots)$$

λ : ($0 \leq \lambda \leq 1$)

これによって、

- ・全ての $\sigma^n(t)$ を使うことができる。
- ・大きいn対しては、 $\sigma^n(t)$ のみが小さくなる。

新たな問題点： λ

以前としてパラであり最適な値が必要で自分で探さないといけない。でも、N-stepよりは精度よく、計算式も理にかなっている。

=====

後方観測 TD(λ)

ベースはTD 法やN-stepTD 法、TD(λ) と同様

- N-stepTD 法の発展
 - 方策評価のアルゴ
 - MDR は未知
 - データ収集を伴う
 - 状態数が多くても OK

TD (λ) の問題点：

1, 更新が遅い:

TD(λ) の式は、

$$V^{\pi_{\text{new}}}(\text{st}) = V^{\pi}(\text{st}) + \alpha \sigma^{\lambda}(t)$$

$$\sigma^\lambda(t) = (1-\lambda)(\sigma^1(t) + \lambda\sigma^2(t) + \dots)$$

この時、必要なn全てを含めようとする兼ね合いで、全てのNの情報が決まらなと計算ができない。→1エピソードの終了まで待つ必要あり。

2,計算が大変：

エピソードが終わっても、全てのNの $\sigma^1(t), \sigma^2(t) \dots$ 計算しその後、全てのN $V^\pi(s_0), V^\pi(s_1), V^\pi(s_2), \dots$ を計算することで更新できる。

この2つの解決策が、後方観測TD法

これまで、エピソードが完了するまで、更新を完了できなかったが、エピソードの中で、各step毎の更新に必要なデータが揃ったタイミングで、更新をしていく。また、この後にてくる、eligibility traceを用いることで、更に計算を省略できる。

=====

eligibility trace

後方観測TD法において、エピソードの終了を待たずに更新を行う方法について触れた。ここから、eligibility traceによって、過去の軌跡をリアルタイムに更新し続けることで、エピソードの終了を待たずに、全てのN-step分の情報を即座に反映させることができる。

計算式： $V^{\pi_{new}}(s) = V^{\pi_{new},(t-1)}(s) + \alpha \sum (\lambda \gamma)^t$

$\sum (\lambda \gamma)^t$ ：この部分がeligibility trace

漸化式：

$Z_{t-1}(s) = 0$. 一番最初の状態

$X_t(s) = \lambda \gamma Z_{t-1}(s) + 1$ (s = s_t の時) or $\lambda \gamma Z_{t-1}(s)$

→eligibility traceは、後方観測TD法においての計算の1部

=>これにより、深層強化学習へ

ここまでの手法(Q学習やTD法)は、「状態(s)を行列のインデックス(番号)」として扱っていた。しかし、囲碁、将棋、あるいはチャットボットのような現実の問題では、「状態」の数が無限に存在するため、すべての $V(s)$ や $Q(s,a)$ を表形式で持っておくことは不可能。

=====

深層強化学習：

=====

深層強化学習アルゴ1：

Policy Gradientの概要

概要：深層強化学習における、方策をアップデートする方法

ポイントとなる数式： $\nabla f = E[\nabla \log \pi(A|S)(Q(S,A) - b(S))]$

今回の背景

- ・ 深層強化学習において、方策 π のパラメータ θ の更新方向を計算
- ・ パラメータ θ の更新に行動価値関数 $Q(s, a)$ を利用
- ・ 今回： Q が既知する
- ・ 今後、 Q が未知のとき、それを推定いく。その方法が今後出てくる REINFORCE や Actore-Gritic の 2 大手法がある

=> 深層強化学習の基礎

状況

- ・ $\pi(a | s) = \pi_\theta(a | s)$ ：方策に深層学習の関数を利用。
- 画像では ResNet など、分野でのアルゴ
 - ・ $f(\theta)$ ：方策 π_θ の収益
- θ を決めると方策 π_θ が決まり、収益が決まる。
 - すなわち、収益はパラメータの関数
 - ・ ∇f の計算
- いつもの勾配の計算のように、 $f(\theta)$ が θ の関数であるため、 f を θ で微分
 - これで θ を更新すると収益 f が大きい θ を獲得できる。
 - => 収益の大きい方策の獲得

ここで用いられるのが、Policy Gradient (方策勾配) 定理
 $\nabla f = E[\nabla \log \pi(A|S)(Q(S,A) - b(S))]$ となる。

★ $\nabla f = E[\nabla \log \pi(A|S)(Q(S,A) - b(S))]$ の深掘り

- ・ この式でやっていることは、いい行動の確率を増やして悪い行動の確率を減らす。
- 結果：方策がどんどんなる。

数式の深掘りにおける登場人物

$f = f(\theta)$ ：方策 π_θ での収益

θ ：方策のパラメータ

$\theta_{\text{new}} = \theta + \alpha \nabla f$ とすると収益増

$\nabla f = E[\nabla \log \pi(A|S)(Q(S,A) - b(S))]$

$E[\dots]$ ： S, A での期待値

実装では、 $\gamma^t \nabla \log \pi_\theta(a_i | s_i)(Q(s_i, a_i) - b(s_i))$ で近似させる。

- ・ $\nabla \log \pi(A|S)(Q(S,A) - b(S))$ とは？

① $\nabla \log \pi(A|S)$ ： θ の関数 $\log \pi(A|S)$ の勾配

② $Q(s, a)$ ：行動価値関数

③ $b(s)$ ：ベースラインとれる関数。分析者が勝手に決めて OK。状態 s での $Q(s, a)$ の平均に設定ことが多い。

深掘りのポイント (1)

$\nabla \log \pi(A|S)(Q(S,A) - b(S))$ の符号に注目

$\theta_{\text{new}} = \theta + \alpha \nabla f$ 、

$\nabla f = E[\nabla \log \pi(A|S)(Q(S,A) - b(S))]$ より

： $\nabla \log \pi(A|S)(Q(S,A) - b(S))$ が正の場合：

θ_{new} は θ より大きい。すなわち

$\nabla \log \pi(A|S) > 0$ かつ $(Q(S,A) - b(S)) > 0$

この時、 $\nabla \log \pi(A|S) > 0$ は、行動 a の確率が増える。また、 $(Q(S,A) - b(S))$ は、行動 a は平均より価値が大きい。こととなる。

また、

$\nabla \log \pi(A|S) < 0$ かつ $(Q(S,A) - b(S)) < 0$

この時、 $\nabla \log \pi(A|S) < 0$ は行動 a の確率が減る。また、 $(Q(S,A) - b(S))$ は、行動 a は平均より価値が低い。こととなる

=> 結果として、いい行動が増えて悪い行動が減る。

： $\nabla \log \pi(A|S)(Q(S,A) - b(S))$ が負の場合：

正の場合同様に、平均よりいい行動の選択確率が増えて、平均より悪い行動の選択確率が減る。

=> 結果。いい方策が得られる。

=====

強化学習アルゴ2: REINFORCE

Policy Gradient の進化系で方策ベースのアルゴ

REINFORCE の1番ベース

$\nabla X = E[\cdot \cdot \cdot]$

$\nabla X \sim 1/n \sum (\cdot \cdot \cdot)$ ：期待値はデータいっぱいにとって平均で近似

これをまえ一言で言えば REINFORCE は、方策勾配 $Q_\pi(S, A)$ g_t で近似する手法

・使う状況

データ ($s_0, a_0, r_0, s_1, a_1, r_1, s_2, a_2, r_2 \cdot \cdot \cdot$: episode データ) の収集はできた。

次に方策勾配を用いて方策 π のパラメータ θ を更新したい。

【↑の使う状況を踏まえた詳細】

大元の計算式は、Policy Gradient の $\nabla f = E[\nabla \log \pi(A|S)(Q(S,A) - b(S))]$

この式における $Q(S,A)$ がわからないため、いっぱいデータを集めたものから出た収益 g_t で近似せる。

すなわち、

$\nabla f \sim 1/N \sum \gamma^t \nabla \log \pi(A|S) g_t - b(S)$

そうすると、

$\theta_{\text{new}} = \theta + \alpha 1/N \sum \gamma^t \nabla \log \pi(A|S) g_t - b(S)$ とすればいい。

ここでベースライン $b(S)$ について

ベースライン $b(S)$ は、理論上なんでも OK だった。でも、実戦では状態 s

$Q(s, a)$ の期待値の方が望ましかった。その為今回は、 $b(S) = V^\pi \theta(s) = \sum \pi(a | s) Q^\pi(s, a)$ の状態価値関数で表される。

そして、この状態価値関数は推定値となり、深層学習の関数なる。

よって

$$\nabla f \sim = 1/N \nabla \sum \gamma^t \nabla \log \pi(A|S) (g_t - V^\pi(s_t))$$

まとめ

Actor-Critic は、データ集めて方策を学習

=====

深層強化学習アルゴ3 : Actor-Critic(A2C)

これも REINFORCE 同様に、Policy Gradient の方策勾配法の 2 大応用の一つ。

概要

- ・ 深層強化学習と方策勾配の手法
- 深層強化学習 : 「データ収集を伴う」, 「MDP は未知」, 「状態数が多くても OK」
 - ・ 方策 π_θ と状態価値関数 V^π を学習
 - ・ TD が出てくる。
 - ・ n-step, λ 版, eligibility trace へ発展可能 ← ここは次回

【復習】

(1) : 一番の基盤 : Policy Gradient

$$\nabla f = E[\nabla \log \pi(A|S) (Q(S,A) - b(S))]$$

(2) : REINFORCE : (1) を扱えるように変形する。

変形① : 期待値を平均で近似

$$E[\dots] \sim = 1/n \sum$$

変形② : $Q(S, A)$ を g_t で置き換え

$$Q(S, A) = g_t = r(t) + \gamma r(t+1) + \gamma^2 r(t+2) \dots$$

変形③ : ベースライン $b(s)$ を V^π で置き換え (収益 r の期待値 = 価値関数 (Q とか V))

$b(s)$ は理論上なんでも良いが、平均に方が学習が安定する。ここでの Q の平均は V^π となるため、 V^π で変える。

Policy Gradient をもとに、変形①、②、③を行うと

$$\nabla f \sim = 1/N \nabla \sum \gamma^t \nabla \log \pi(A|S) g_t - V^\pi(s_t) \text{ となる}$$

(3) : Actor-Critic : (2) の REINFORCE の式 $\nabla f \sim = 1/N \nabla \sum \gamma^t \nabla \log \pi(A|S) (g_t - V^\pi(s_t))$ に変化をさらに加える。

変形① : g_t を $\gamma V^\pi(s_{t'})$ で置き換える。

$g_t = r(t) + \gamma r(t+1) + \gamma^2 r(t+2) \dots$ より、これは収益に割引現在価値
与えた、報酬の期待値であるため、 $g_t = r(t) + \gamma V^\pi(s_{t'})$ と置き換えられる。すると、
 $\nabla f \sim = 1/N \nabla \sum \gamma^t \nabla \log \pi(A|S) (r(t) + \gamma V^\pi(s_{t'}) - V^\pi(s_t))$ となる。

変形② : $r(t) + \gamma V^\pi(s_{t'}) - V^\pi(s_t)$ を $\sigma_t(TD)$: TD 誤差で置き換える。

① で変化を加えた $r(t) + \gamma V^\pi(s_{t'}) - V^\pi(s_t)$ の部分は、TD 誤差の式となるため、置き換えて変形させると、

$$\begin{aligned} \nabla f \sim &= 1/N \nabla \sum \gamma^t \nabla \log \pi(A|S) (r(t) + \gamma V^\pi(s_{t'}) - V^\pi(s_t)) \\ &= 1/N \sum \gamma^t \sigma_t(TD) \nabla \log \pi(a_t | s_t) \end{aligned}$$

これを用いて、 θ を更新すると、

$$\theta_{\text{new}} = \theta + \alpha \{ 1/N \sum \gamma^t \sigma_t(TD) \nabla \log \pi(a_t | s_t) \}$$

これで、方策の更新ができた。次にこれをもとに最初に挙げた価値関数の更新を考えると
価値関数 (new) = 価値関数 (t) + $\alpha \{1/N \sum \sigma_i(TD) \nabla V^*(s_i)\}$
すればいい α : 学習率

まとめ

Actor-Critic は、データ集めて方策と価値関数を学習

=====

深層強化学習における eligibility trace

前提として、eligibility trace は TD 誤差が出てきた時に計算を簡単にする手法。
TD 誤差とは、 $(r(t) + \gamma V^*(s_{t'}) - V^*(s_t))$ この部分。の数式のは、新しい現実
引く、今までの予測で表されるもの。この新しい現実の
部分がアルゴによってはその時の報酬 g で計算するものがある
この場合、eligibility trace は使えない。(例 : REINFORCE)

=====

強化学習アルゴ 4 : DQN

概要 : Q 学習の深層学習バージョン

- ・ $Q\theta$: 最適行動価値関数を推定器する。
- ・ $Q\theta$ を使って TD 誤差を出す
- ・ $\sigma\theta$: TD 誤差が最小のように学習したい。

すなわち、

$L(\theta) = 1/2 E[(\sigma\theta)^2]$ これを最小にする。

微分をすると、

$\nabla L = -E[\sigma\theta \nabla Q\theta(S, A)]$

$\theta_{\text{new}} = \theta - \alpha \nabla L$

結果として

$\sigma\theta$ の小さい $Q\theta$ が手に入る。← これって、最適行動価値関数だね。

Q 学習をもとに紐解くと

Q 学習は、行動価値関数をベースに、ベルマン最適作用素を使ったものだった。

数式では、

$\sigma\theta = r + \gamma \max_{a'} Q^*(s', a') - Q^*(s, a)$

が小さくなるように学習を進めていった。

$\sigma\theta$: TD 誤差

$r + \gamma \max_{a'} Q^*(s', a')$: 正しいと思っているもの (正解ラベルのようなもの)

$Q^*(s, a)$: 最適方策 π^* の行動価値関数 Q^* の推定値

Q 学習における $\sigma\theta$: TD 誤差を小さくしようとするモチベーションを、 $L(\theta) = 1/2E[(\sigma\theta)^2]$ を小さくしようとするモチベーションで置き換えたものが、DQN 結果として、

$\sigma\theta$ の小さい $Q\theta$ が手に入る。← これって、最適行動価値関数だよな。

詳細 : target network

DQN の数式である

$$\sigma\theta = r + \gamma \max_{a'} Q^*(s', a') - Q^*(s, a)$$

について考える。

ポイントは、 $r + \gamma \max_{a'} Q^*(s', a')$ と $Q^*(s, a)$ の差をなくすこと。

ただ、ここでの注意点が、本来 $r + \gamma \max_{a'} Q^*(s', a')$ を正解として、これに向かって、

$Q^*(s, a)$ を更新していきたいが、このまま実装すると、 $Q^*(s, a)$ に向かって、

$r + \gamma \max_{a'} Q^*(s', a')$ が更新されていき、悪い学習になることもある。

これを解消するための工夫 ↓

$$\sigma\theta = r + \gamma \max_{a'} Q^{(\theta-1)}(s', a') - Q^{\theta i}(s, a) \text{ を用いる。}$$

そして、 θ の更新では、

$$\theta(l+1) = \theta i - 1/2\alpha E[\nabla \theta i(\sigma\theta(i-1), \theta i)] \text{ とする。}$$

ポイントは、 $\nabla \theta i$ とすることで、 $\sigma\theta = r + \gamma \max_{a'} Q^{(\theta-1)}(s', a') - Q^{\theta i}(s, a)$ における、 $Q^{\theta i}(s, a)$ だけ更新するようにする。

この時、正解ラベルのような役割をする、 $Q^{(\theta-1)}$ を target network という。

↑ 実装で大事。

=====

